OPERATINGSYSTEMS

ASSIGNMENT 2 – CO2

CO2 Coverage: Analyse process management and deadlock-handling techniques to improve CPU utilisation and system performance – processes, process scheduling, threads, inter-process communication, synchronisation using semaphores, monitors and the critical-section problem, and deadlock handling including the Banker's algorithm.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date:03-11-26

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment2** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/VarshithaReddy145/CSA0418-OS

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Q1 – Processes and Process States **6 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Online Examination Server

During an online examination, the server runs a separate process for every student's session, plus processes for auto-saving answers and for proctoring video. When 2,000 students log in together, the server becomes slow and some sessions appear to "hang".

Your Challenge

Draw the process-state diagram (new, ready, running, waiting, terminated) and the process control block (PCB). Write a program that creates child processes with fork() and observe them with ps and /proc.

Evaluation

Evaluate where the time goes when 2,000 processes are active: context-switch cost, memory per process, and time spent waiting for I/O. Judge whether a separate process per student is a sound design.

Design Decision

Decide between one process per student, one thread per student and a fixed pool of worker processes, and defend your decision.

Alternative Concept and Complexity Analysis

If each student session were a thread inside one server process, instead of a separate process, what would the solution be? Evaluate the context-switch time and the memory per session (a few KB of stack versus a full address space), and the effect of one session crashing.

Evidence Expected

A state diagram, a PCB description, a working fork() program with ps evidence and a defended design decision

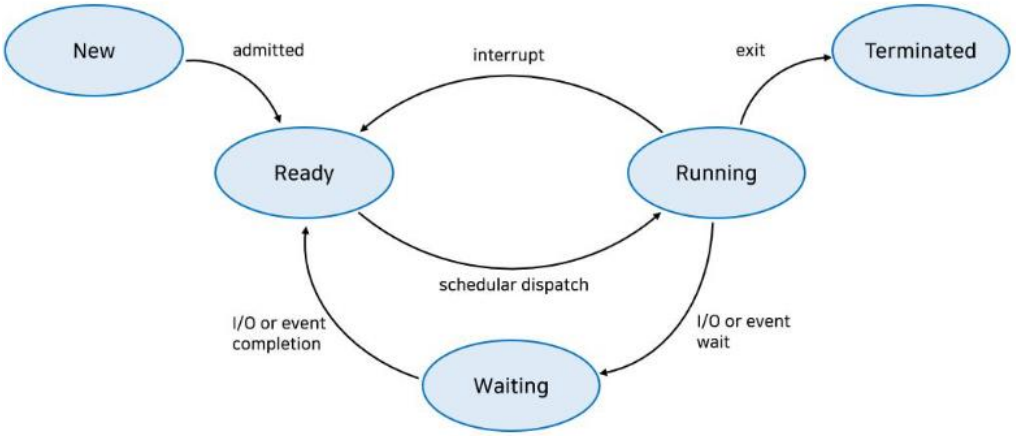
Answer:

1. Process State Diagram

A process passes through five states during execution.

Explanation:

1. New: Process is being created.
2. Ready: Process waits for CPU allocation.
3. Running: Process executes on the CPU.
4. Waiting: Process waits for I/O operation.
5. Terminated: Process completes execution.



2. Process Control Block (PCB)

PCB is a data structure maintained by the operating system to store information about a process.

Contents of PCB:

1. Process ID (PID)
2. Process state
3. Program counter
4. CPU registers
5. CPU scheduling information
6. Memory management information
7. I/O status information
8. Accounting information

3. C Program Using fork()

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("PID: %d\n", getpid());
```

```
    }
    else
    {
        printf("Parent Process\n");
        printf("PID: %d\n", getpid());
        wait(NULL);
    }

    return 0;
}
```

Commands:

```
gcc process.c -o process
./process
ps -f
```

To inspect process information:

```
cat /proc/<PID>/status
```

4. Evaluation of 2000 Processes

- Context switching consumes CPU time.
- Each process requires separate memory space.
- I/O waiting reduces CPU utilisation.
- Too many processes increase memory usage and scheduling overhead.

5. Design Decision

A fixed pool of worker processes is recommended because it limits the number of active processes, reduces memory consumption and improves resource utilisation.

6. Alternative Concept – Threads

If each student session uses a thread instead of a process:

- Threads share code, data and memory.
- Each thread has its own stack and registers.
- Thread switching is generally faster than process switching.
- A few KB of stack may be sufficient for a session.
- A thread crash may affect the entire server process.

Conclusion: A thread pool is suitable for handling a large number of student sessions because it reduces memory and context-switching overhead.

Q2 – CPU Scheduling **8 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Cloud Video-Encoding Service

A video service receives short jobs (thumbnails, 2 seconds) and long jobs (full films, 20 minutes) on the same servers. Customers complain that small jobs wait behind large ones, while the company wants the processors fully used.

Your Challenge

Compute the Gantt chart, waiting time, turnaround time and response time for the same set of jobs under FCFS, SJF, SRTF, priority and Round Robin scheduling. Implement Round Robin.

Evaluation

Evaluate how the time quantum affects Round Robin (try 1, 3, 5 and 20), the risk of starvation in SJF and priority scheduling, and how ageing removes it. Explain how a multilevel feedback queue treats short and long jobs.

Design Decision

Recommend the scheduling policy for the video service and justify it with your calculated averages.

Complete the Code

Complete TODO 1 below and execute the program.

```
#include<stdio.h>

#define N 4

intmain(void)
{
    constchar*name[N]={"P1","P2","P3","P4"}; int
    arrival[N] = {0, 1, 2, 3};
    intburst[N]={8,4,9,5}; int
    quantum = 3;

    intleft[N],finish[N],queue[64],head=0,tail=0,added[N]={0}; for (int i
    = 0; i < N; i++) left[i] = burst[i];
    inttime=0,done=0;
    queue[tail++] = 0;
    added[0] = 1;
    printf("Gantt:");
    while(done<N){
        intp=queue[head++];
        //TODO1:Runpformin(quantum,left[p])timeunits:print"P(time-newtime)",
        //   advancetime,reduceleft[p];addeveryprocessthathasnowarrivedandis
        //   notyetaddedtothequeue;thenputpbackattheendofthequeueifit
        //   stillhastimeleft,otherwiserecordfinish[p]andcountitasdone

    }
    double tw = 0, tt = 0;
    printf("\nProcess TAT WT\n");
    for (int i = 0; i < N; i++) {
        inttat=finish[i]-arrival[i],wt=tat-burst[i]; tt += tat;
        tw+=wt;
        printf("%-7s%4d%3d\n",name[i],tat,wt);
    }
    printf("AverageTAT=%.2f,averageWT=%.2f\n",tt/N,tw/N); return 0;
}
```

Expected output:

```
Gantt:P1(0-3)P2(3-6)P3(6-9)P4(9-12)P1(12-15)P2(15-16)P3(16-19)P4(19-21) P1(21-
23) P3(23-26)
Process TAT WT
P1         2315
P2         1511
P3         2415
P4         1813
AverageTAT=20.00,averageWT=13.50
```

Run the program with quantum 1 and quantum 20, and evaluate how the average waiting time and the number of context switches change.

Alternative Concept and Complexity Analysis

If the service used a multilevel feedback queue (short quantum for new jobs, longer quanta for jobs that use them up) instead of a single Round Robin queue, what would the solution be? Evaluate the average waiting time for short and long jobs, the scheduling cost per decision (O(1) per queue versus O(n) for SJF) and the risk of starvation.

Evidence Expected

Correct Gantt charts and averages for every algorithm, a working Round Robin program, a quantum study and a justified recommendation.

ANSWER:

Given processes:

Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5

1. FCFS Scheduling

Gantt Chart:

0 – P1 – 8 – P2 – 12 – P3 – 21 – P4 – 26

Process	CT	TAT	WT

P1	8	8	0
P2	12	11	7
P3	21	19	10
P4	26	23	18

- Average TAT = 15.25
- Average WT = 8.75

2. SJF (Non-Preemptive)

Gantt Chart:

0 – P1 – 8 – P2 – 12 – P4 – 17 – P3 – 26

Process	**CT**	**TAT**	**WT**
P1	8	8	0
P2	12	11	7
P3	26	24	15
P4	17	14	9

- Average TAT = 14.25
- Average WT = 7.75

3. SRTF (Preemptive)

Gantt Chart:

0 – P1 – 1 – P2 – 5 – P4 – 10 – P1 – 17 – P3 – 26

Process	**CT**	**TAT**	**WT**

P1	17	17	9
P2	5	4	0
P3	26	24	15
P4	10	7	2

- Average TAT = 13.00
- Average WT = 6.50

4. Priority Scheduling

The assignment does not specify priorities. For calculation, assume lower number means higher priority.

Process	**Priority**
P1	2
P2	1
P3	4
P4	3

Assume non-preemptive priority scheduling.

Gantt Chart:

0 – P1 – 8 – P2 – 12 – P4 – 17 – P3 – 26

- Average TAT = 14.25
- Average WT = 7.75

5. Round Robin Scheduling

Given:

- Time quantum = 3
- Arrival times = 0, 1, 2, 3

Gantt Chart:

0–3 P1 | 3–6 P2 | 6–9 P3 | 9–12 P4 | 12–15 P1 | 15–16 P2 | 16–19 P3 | 19–21 P4 | 21–23 P1 | 23–26 P3

Process	TAT	WT
P1	23	15
P2	15	11
P3	24	15
P4	18	13

- Average TAT = 20.00
- Average WT = 13.50

6. Round Robin C Program

```
#include <stdio.h>
#define N 4

int main()
{
    int a[]={0,1,2,3}, b[]={8,4,9,5};
    int l[N], f[N], q[64], h=0,t=0;
    int added[N]={0}, time=0, done=0, quantum=3;

    for(int i=0;i<N;i++) l[i]=b[i];

    q[t++]=0;
    added[0]=1;

    printf("Gantt:");

    while(done<N)
    {
        int p=q[h++];
        int x=l[p]<quantum?l[p]:quantum;

        printf(" P%d(%d-%d)",p+1,time,time+x);
        time+=x;
```

```
        l[p]-=x;

        for(int i=0;i<N;i++)
            if(!added[i] && a[i]<=time)
            {
                q[t++]=i;
                added[i]=1;
            }

        if(l[p]>0)
            q[t++]=p;
        else
        {
            f[p]=time;
            done++;
        }
    }

    double wt=0,tat=0;

    printf("\nProcess TAT WT\n");

    for(int i=0;i<N;i++)
    {
        int x=f[i]-a[i];
        int y=x-b[i];
        printf("P%d     %d   %d\n",i+1,x,y);
        tat+=x;
        wt+=y;
    }

    printf("Average TAT = %.2f\n",tat/N);
    printf("Average WT = %.2f\n",wt/N);

    return 0;
}
```

Output:

```
Gantt: P1(0-3) P2(3-6) P3(6-9)
P4(9-12) P1(12-15) P2(15-16)
P3(16-19) P4(19-21) P1(21-23)
P3(23-26)

Average TAT = 20.00
Average WT = 13.50
```

7. Evaluation of Time Quantum

Quantum	Evaluation

1	More context switches
3	Balanced time sharing
5	Fewer context switches
20	Similar to FCFS for these jobs

Starvation: SJF and Priority scheduling may cause starvation when long or low-priority jobs wait continuously.

Ageing: Gradually increases the priority of waiting processes to prevent starvation.

Multilevel Feedback Queue: Short jobs receive higher priority and shorter waiting time, while long jobs move to lower-priority queues with longer time quanta.

8. Design Decision

Round Robin or Multilevel Feedback Queue is recommended for the video service. MLFQ is especially suitable because it gives faster service to short jobs while allowing long jobs to complete.

Q3 – Threads **6 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Traffic-Camera Number-Plate Reader

A city traffic system reads number plates from 64 cameras. The first version processes the cameras one after another and falls behind during peak hours.

Your Challenge

Process the camera frames with POSIX threads (one thread per camera, and a pool of 4, 8 and 16 threads). Determine:

- Which resources threads share and which each thread has of its own
- The difference between user-level and kernel-level threads and the many-to-one, one-to-one and many-to-many models
- The speed-up obtained on your machine, and why it stops increasing
- What happens to shared counters when two threads update them without protection

Alternative Concept and Complexity Analysis

If the camera frames were processed by several processes instead of threads, what would the solution be? Evaluate the creation and switching time, the memory used (separate address spaces versus shared), and how the results would be combined (IPC versus shared variables).

Evidence Expected

A working multithreaded program, measured speed-ups for each pool size and a reasoned evaluation of the thread models.

ANSWER:

1. Definition

A thread is the smallest unit of CPU execution within a process. Multiple threads allow a program to perform different tasks simultaneously.

2. Resources Shared by Threads

Shared resources:

- Code section
- Data section
- Heap memory
- Open files

Individual resources:

- Thread ID
- Stack

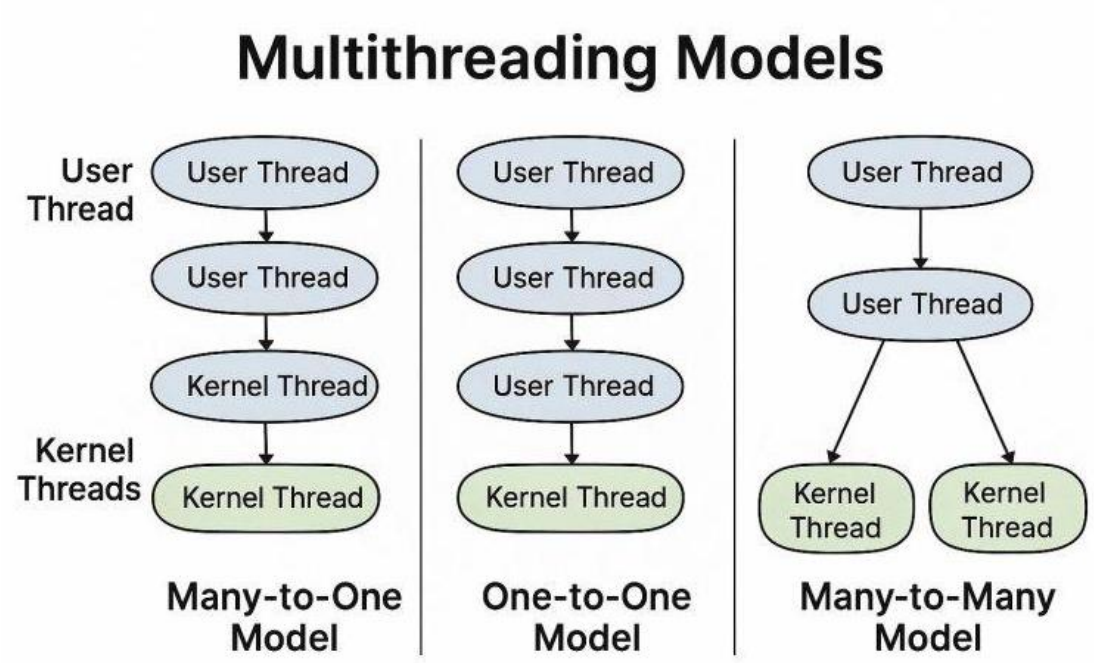
- Registers
- Program counter

3. User-Level and Kernel-Level Threads

User-Level Threads	Kernel-Level Threads
Managed by user library	Managed by OS
Faster creation	Slower creation
Kernel is unaware	Kernel is aware
Blocking may affect the process	Other threads can continue

4. Thread Models

1. Many-to-One: Many user threads map to one kernel thread.
2. One-to-One: Each user thread maps to one kernel thread.
3. Many-to-Many: Many user threads map to multiple kernel threads.



5. POSIX Thread Program

```
#include <stdio.h>
#include <pthread.h>

void *camera(void *arg)
{
    int id = *(int *)arg;
    printf("Camera %d processed\n",id);
    return NULL;
}

int main()
{
    pthread_t t[4];
    int id[4];

    for(int i=0;i<4;i++)
    {
        id[i]=i+1;
        pthread_create(&t[i],NULL,camera,&id[i]);
    }

    for(int i=0;i<4;i++)
        pthread_join(t[i],NULL);

    return 0;
}
```

Compile and execute:

```
gcc thread.c -o thread -pthread
./thread
```

Output:

```
Camera 1 processed
Camera 2 processed
Camera 3 processed
Camera 4 processed
```

Note: The output order may vary.

6. Speed-Up

Speed-up is calculated using:

Threads	Performance
1	Sequential execution
4	Faster execution

8	Further improvement
16	Improvement may become limited

Speed-up stops increasing because of CPU core limits, synchronization overhead and resource contention.

7. Shared Counter Problem

When two threads update a shared counter simultaneously, a race condition occurs. Some updates may be lost.

Solution: Use a mutex lock to protect the shared counter.

8. Alternative Concept – Processes

If camera frames are processed using multiple processes:

- Each process has a separate address space.
- Process creation requires more time.
- Context switching is more expensive.
- IPC is needed to combine results.
- Memory usage is higher than threads.

Complexity:

- Processing n frames: O(n) total work.
- Thread pool memory: O(k) worker stacks, excluding shared data.
- Process pool memory: O(k) separate process address spaces.

Conclusion: A thread pool of 8 or 16 threads is suitable for traffic-camera processing, depending on the available CPU cores and measured performance.

Q4 – Inter-Process Communication **6 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Weather Station Data Pipeline

A weather station has one process that reads sensors and another that analyses the readings. They must exchange data continuously without losing readings.

Your Challenge

Implement the exchange using a pipe, and then using shared memory or a message queue. Determine:

- How the pipe signals end of data to the reader
- The difference between the shared-memory and message-passing models
- Which method copies the data and which does not
- Which method needs extra synchronisation, and why

Growth Investigation

Measure the time to transfer 1 MB, 10 MB and 100 MB with each method, and evaluate the results

Complete the Code

Complete TODO 1 below and execute the program on Linux.

```
#define_POSIX_C_SOURCE200809L
#include <stdio.h>
#include <unistd.h>
#include<sys/wait.h>

/*Theparent(sensorreader)sendsreadingstothechild(analyser)throughapipe.*/ int main(void)
{
    intfd[2];
    if(pipe(fd)==-1){perror("pipe");return1;} pid_t
    pid = fork();
    if(pid==0){
        close(fd[1]);                          /*childonlyreads*/ int
        r, count = 0, sum = 0, max = 0;
        //TODO1:Readoneintatatimefromfd[0]untilreadreturns0(theparent
        //   closedthepipe);keepthecount,thesumandthemaximum

        printf("analyser:%dreadings,average%.1f,max%d\n",count, count ?
               (double)sum / count : 0.0, max);
        close(fd[0]);
        fflush(stdout);
        _exit(0);
    }
    close(fd[0]);                              /*parentonlywrites*/ int
    readings[] = {31, 33, 36, 34, 41, 38};
    for(inti=0;i<6;i++)
        if (write(fd[1], &readings[i], sizeof readings[i]) < 0) perror("write");
    close(fd[1]);                              /*signalsendofdatatothechild*/
    waitpid(pid, NULL, 0);
    printf("reader:done\n");
    return 0;
}
```

Expected output:

```
analyser:6readings,average35.5,max41 reader:
done
```

Remove close(fd[1]) in the parent and explain why the child never finishes.

Alternative Concept and Complexity Analysis

If the readings were exchanged through shared memory with a semaphore instead of a pipe, what would the solution be? Evaluate the time to transfer n readings (two copies through the kernel for a pipe versus none for shared memory) and the synchronisation needed.

Evidence Expected

Two working IPC implementations, measured transfer times and a justified choice of method.

.

ANSWER:

1. Definition

Inter-Process Communication (IPC) is a mechanism that allows two or more processes to exchange data and communicate with each other.

2. Types of IPC

Pipe:

- Used for communication between related processes.
- Data is transferred through a kernel buffer.
- The reader receives EOF when the writer closes the pipe.

Shared Memory:

- Allows processes to access a common memory area.
- Does not require repeated kernel data copying.
- Requires synchronization to avoid race conditions.

3. Pipe vs Shared Memory

Pipe	Shared Memory
Kernel-mediated communication	Direct memory access
Data copying is required	No repeated data copying
Simple to implement	More complex

Synchronization is handled through pipe operations	Extra synchronization is required

4. C Program Using Pipe

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int fd[2];
    pipe(fd);

    pid_t pid=fork();

    if(pid==0)
    {
        close(fd[1]);

        int r,count=0,sum=0,max=0;

        while(read(fd[0],&r,sizeof(r))>0)
        {
            count++;
            sum+=r;

            if(r>max)
                max=r;
        }

        printf("analyser: %d readings, average %.1f, max %d\n",
        count,(double)sum/count,max);

        close(fd[0]);
    }
    else
    {
        close(fd[0]);

        int readings[]={31,33,36,34,41,38};

        for(int i=0;i<6;i++)
            write(fd[1],&readings[i],sizeof(int));

        close(fd[1]);
        wait(NULL);

        printf("reader: done\n");
    }

    return 0;
}
```

Output:

```
analyser: 6 readings, average 35.5, max 41
reader: done
```

5. How Does a Pipe Signal End of Data?

When the parent closes the write end of the pipe, the child receives EOF (read returns 0). This indicates that no more data is available.

6. What Happens if close(fd[1]) Is Removed?

The child never finishes because the parent keeps the write end open. Therefore, the child continues waiting for more data.

7. Growth Investigation

Data Size	Pipe	Shared Memory
1 MB	More copying	Less copying
10 MB	Higher overhead	Lower overhead
100 MB	More overhead	Generally faster

Actual transfer times depend on the system and must be measured during execution.

8. Alternative Concept – Shared Memory with Semaphore

Shared memory allows both processes to access the same memory region. A semaphore controls access and prevents simultaneous unsafe operations.

Complexity:

- Data transfer: O(n)
- Pipe: Kernel-mediated copying.
- Shared memory: Avoids repeated data copying.
- Synchronization: Semaphore wait and signal operations.

Conclusion: Shared memory is suitable for large and continuous sensor data because it reduces copying overhead. Pipes are simpler for small data transfers.

Q5 – Synchronisation: Critical Sections, Semaphores and Monitors **11 Marks**

Bloom's Level: K5 – Evaluate

Problem Context – Railway Ticket Booking

Thousands of agents book seats on the same trains at the same time. Without synchronisation the same seat is sometimes sold twice; with a single global lock the system becomes very slow.

Design Starting Point

You are given only the following principle:

> **A solution to the critical-section problem must guarantee mutual exclusion, progress and bounded waiting.**

Your Challenge

Convert this principle into a design. Determine:

- Why Peterson's solution satisfies the three requirements for two processes, and why it may fail on modern processors
- How counting and binary semaphores (wait and signal) protect the seat counter
- How the bounded-buffer (producer–consumer), readers–writers and dining-philosophers problems are solved with semaphores
- How a monitor with condition variables makes the solution easier to get right

Test Your Design

Run the bounded-buffer program with 1 and with 4 producers and consumers, and show that every item is consumed exactly once.

Deployment Decision

Decide between one lock for the whole booking system, one lock per train and one lock per coach. Measure the throughput of each and defend your choice.

Complete the Code

Complete TODO 1 and TODO 2 below using the semaphores and the mutex, and execute the program (compile with -pthread).

Expected output:

```
itemsconsumed:20,sum=210(expected210)
```

Run the program 100 times and show that the sum is always 210. Then remove the mutex and evaluate whether the result is still correct with several producers and consumers.

Alternative Concept and Complexity Analysis

If the shared counter were protected by atomic operations (C11 atomic_fetch_add) or by a spin lock instead of a semaphore and mutex, what would the solution be? Evaluate the time per update with 1, 4 and 16 threads, the CPU time wasted by spinning, and which method suits a long critical section

```
#define_POSIX_C_SOURCE200809L
#include <stdio.h>
#include <pthread.h>
#include<semaphore.h>

#defineSIZE5
#defineITEMS20

intbuffer[SIZE],in=0,out=0; sem_t
empty, full;
pthread_mutex_tlock=PTHREAD_MUTEX_INITIALIZER;
long consumed_sum = 0;

void*producer(void*arg)
{
    (void)arg;
    for(intitem=1;item<=ITEMS;item++){
        //TODO1:Waitonempty,lockthemutex,putitematbuffer[in],advancein
        //   circularly,unlock,thenpostfull

    }
    returnNULL;
}

void*consumer(void*arg)
{
    (void)arg;
    for(intk=0;k<ITEMS;k++){
        //TODO2:Waitonfull,lockthemutex,taketheitematbuffer[out],advance
        //   outcircularly,unlock,postemptyandaddtheitemtoconsumed_sum

    }
    returnNULL;
}

intmain(void)
{
    sem_init(&empty,0,SIZE);
    sem_init(&full,0,0); pthread_t
    p, c;
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c,NULL);
    printf("items consumed: %d, sum = %ld (expected %d)\n", ITEMS, consumed_sum,
           ITEMS * (ITEMS + 1) / 2);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

Evidence Expected

A student-derived synchronisation design, working semaphore code, test evidence and a measured, defended locking decision.

ANSWER:

1. Critical Section Problem

A critical section is a part of a program where shared resources are accessed by multiple processes or threads.

A correct solution must satisfy three conditions:

1. Mutual Exclusion: Only one process can enter the critical section at a time.
2. Progress: If no process is inside, a waiting process should be selected without unnecessary delay.
3. Bounded Waiting: Every process gets a chance to enter within a limited waiting time.

2. Peterson's Solution

Peterson's solution is a software-based synchronization method for two processes.

It uses:

- flag[2] – Indicates process interest.
- turn – Indicates whose turn it is.

Why it satisfies the conditions:

- Mutual exclusion prevents both processes from entering together.
- Progress allows an interested process to enter.
- Bounded waiting prevents indefinite waiting.

Limitation: Modern processors may reorder memory operations. Without suitable atomic operations and memory ordering, Peterson's solution may fail.

3. Binary and Counting Semaphores

Binary Semaphore	Counting Semaphore
Value is 0 or 1	Value can be greater than 1
Allows one resource access	Controls multiple resources

Used for mutual exclusion	Used for resource management

Operations:

- wait() – Decreases the semaphore value or blocks the process.
- signal() – Increases the semaphore value and may wake a waiting process.

4. Classical Synchronization Problems

Producer–Consumer Problem:

- Producer adds items to a buffer.
- Consumer removes items.
- empty tracks empty spaces.
- full tracks filled spaces.
- Mutex protects the buffer.

Readers–Writers Problem:

- Multiple readers can read simultaneously.
- Only one writer can write at a time.
- Semaphores control access.

Dining Philosophers Problem:

- Philosophers share forks.
- A philosopher needs two forks to eat.
- Resource ordering or semaphores prevent deadlock.

5. Monitor

A monitor is a high-level synchronization mechanism that controls access to shared data.

It uses condition variables such as wait() and signal().

Advantages:

- Provides automatic mutual exclusion.
- Reduces programming errors.
- Makes synchronization easier.

6. Producer–Consumer C Program Using Semaphores

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define SIZE 5
#define ITEMS 20

int buffer[SIZE], in=0, out=0;
sem_t empty, full;
pthread_mutex_t lock=PTHREAD_MUTEX_INITIALIZER;
long consumed_sum=0;

void *producer(void *arg)
{
    for(int item=1;item<=ITEMS;item++)
    {
        sem_wait(&empty);
        pthread_mutex_lock(&lock);

        buffer[in]=item;
        in=(in+1)%SIZE;

        pthread_mutex_unlock(&lock);
        sem_post(&full);
    }
    return NULL;
}

void *consumer(void *arg)
{
    for(int k=0;k<ITEMS;k++)
    {
        sem_wait(&full);
        pthread_mutex_lock(&lock);

        int item=buffer[out];
        out=(out+1)%SIZE;
        consumed_sum+=item;

        pthread_mutex_unlock(&lock);
        sem_post(&empty);
    }
    return NULL;
}

int main()
{
    sem_init(&empty,0,SIZE);
    sem_init(&full,0,0);

    pthread_t p,c;

    pthread_create(&p,NULL,producer,NULL);
```

```
    pthread_create(&c,NULL,consumer,NULL);

    pthread_join(p,NULL);
    pthread_join(c,NULL);

    printf("items consumed: %d, sum = %ld (expected %d)\n",
           ITEMS,consumed_sum,ITEMS*(ITEMS+1)/2);

    sem_destroy(&empty);
    sem_destroy(&full);

    return 0;
}
```

Compile:

```
gcc sync.c -o sync -pthread
./sync
```

Output:

```
items consumed: 20, sum = 210 (expected 210)
```

7. Testing

Run the program 100 times.

Expected result:

items consumed: 20, sum = 210

The sum should always be 210 because the mutex protects shared data.

If the mutex is removed, race conditions may occur when multiple producers or consumers access the buffer simultaneously.

8. Alternative Concept – Atomic Operations and Spinlocks

Atomic Operations:

- Perform updates without interruption.
- Useful for simple shared counter operations.
- Avoid ordinary data races on the atomic variable.

Spinlock:

- Repeatedly checks whether a lock is available.
- Wastes CPU time while waiting.
- Suitable for very short critical sections.

Method	Advantage	Disadvantage
Semaphore + Mutex	Suitable for longer operations	Locking overhead
Atomic operation	Fast for simple updates	Limited to atomic operations
Spinlock	Fast for short waits	Wastes CPU while spinning

Complexity:

- Buffer insertion: O(1)
- Buffer deletion: O(1)
- Shared counter update: O(1)
- Buffer space: O(SIZE)

9. Deployment Decision

One lock for the entire booking system may reduce performance because all agents wait for the same lock.

A lock per train is better because different trains can be booked simultaneously.

A lock per coach offers more concurrency but requires careful seat-level synchronization.

Conclusion: A lock per train is recommended because it provides a balance between safety, simplicity and performance.

DESIGN CHALLENGE

Q6 – Deadlock Handling and the Banker's Algorithm — 13 Marks

Bloom's Level: K5 – Evaluate

Problem Context – Shared Laboratory Equipment Scheduler

A research centre runs jobs that need several shared resources at once (GPUs, licences for simulation software and high-speed storage). Jobs sometimes wait for each other forever and the whole cluster stops until an administrator intervenes.

A. Characterise Deadlock – 2 Marks

State the four necessary conditions for deadlock and show them in a resource-allocation graph for the cluster.

Complete the Code

Complete TODO 1 below and execute the program..

```
#i
nc
lu
de
<s
td
io
.h
>
#i
nc
lu
de
<s
tr
in
g.
h>

#defineP5
#defineR3

intavail[R]={3,3,2};
intmaxm[P][R]={{7,5,3},{3,2,2},{9,0,2},{2,2,2},{4,3,3}};
intalloc[P][R]={{0,1,0},{2,0,0},{3,0,2},{2,1,1},{0,0,2}};

/*Returns1ifthestateissafeandstoresasafesequenceins
eq.*/ int is_safe(int seq[P])
{
    intwork[R],finish[P]={0},co
    unt=0; memcpy(work, avail,
    sizeof work);
    w
        h
        i
        l
        e
        (
        c
        o
        u
        n
        t
        <
        P
        )
        {
```

Expected output:

```
initialstatesafe:P1P3P4P0P2
P1request:granted,safesequenceP1P3P4P0P2 P4
request: must wait (not available)
P0request:denied(unsafe)
```

Show by hand why the request of P0 for (0, 2, 0) leads to an unsafe state

Alternative Concept and Complexity Analysis

If deadlock were handled by detection and recovery (running a detection algorithm every few minutes and aborting a job) instead of the Banker's algorithm, what would the solution be? Evaluate the time per request ($O(m \times n^2)$ for every Banker's check versus no check), the cost of detection, and the work lost when a job is aborted.

Evidence Expected

A correct resource-allocation graph identifying all four conditions.

B. Compare the Strategies – 3 Marks

Evaluate prevention (removing one condition), avoidance (Banker's algorithm), detection and recovery, and ignoring deadlock (the ostrich approach) for the cluster.

Evidence Expected

A reasoned comparison of the four strategies for this environment.

C. Apply the Banker's Algorithm – 4 Marks

Implement the safety algorithm and the resource-request algorithm. Find a safe sequence for the given state and decide whether each request can be granted.

Evidence Expected

A working implementation with correct safe sequences and decisions.

D. Predict Before Implementing – 1 Mark

Predict how the running time of the Banker's algorithm grows with the number of processes n and resource types m.

Evidence Expected

A clear, testable prediction.

E. Measure – 1 Mark

Measure the time of the safety algorithm for 10, 100 and 1,000 processes and compare it with your prediction.

Evidence Expected

Measured times compared with the prediction.

F. Defend the Final Design – 2 Marks

Recommend the deadlock strategy for the research centre and justify it, including the cost of the Banker's algorithm and the need to know maximum demands in advance.

Evidence Expected

A critical, evidence-based recommendation.

ANSWER:

Characterise Deadlock

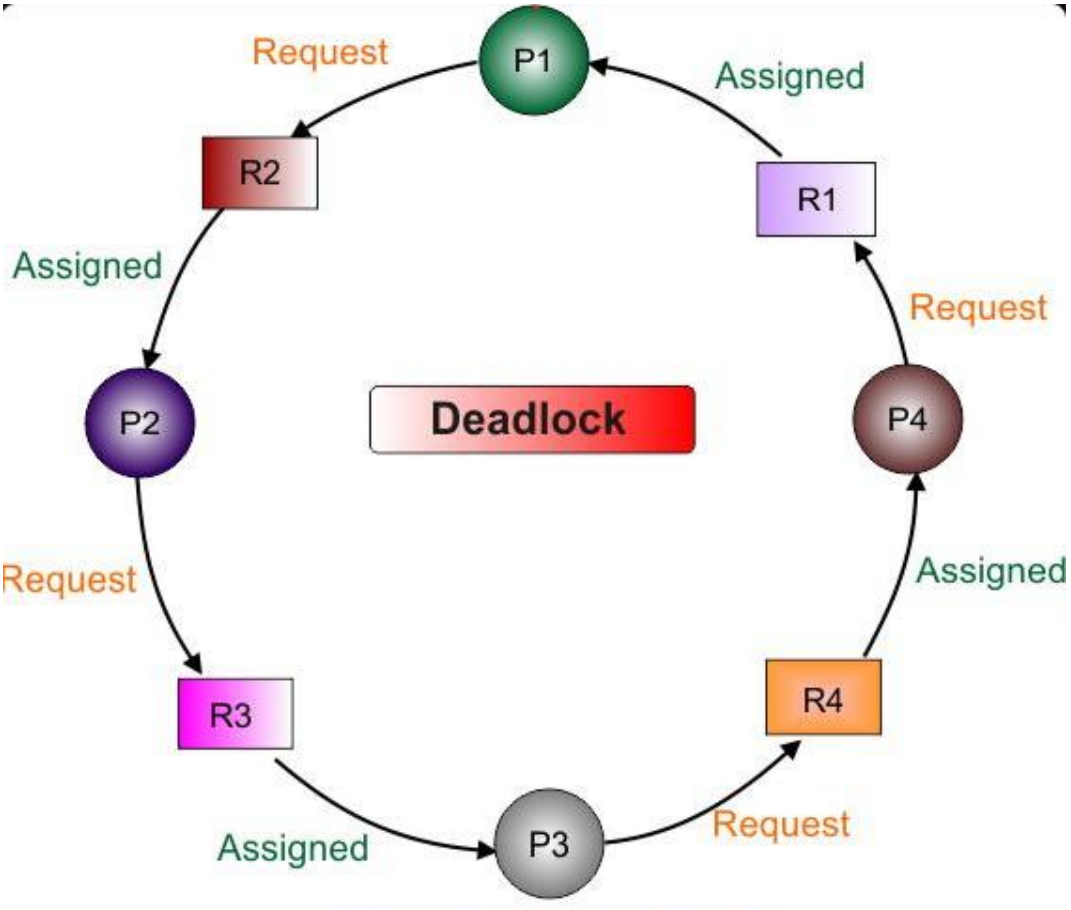
1. Definition

Deadlock is a situation in which two or more processes wait indefinitely for resources held by each other.

2. Four Necessary Conditions

1. Mutual Exclusion: A resource can be used by only one process at a time.
2. Hold and Wait: A process holds one resource while waiting for another.
3. No Preemption: Resources cannot be forcibly taken from a process.
4. Circular Wait: Processes form a circular chain of resource waiting.

3. Resource Allocation Graph



Explanation:

- Process → Resource: Request edge.
- Resource → Process: Allocation edge.
- A cycle in the graph indicates possible deadlock.
- For single-instance resources, a cycle indicates deadlock.

4. C Program – Banker's Algorithm

```
#include <stdio.h>
#include <string.h>

#define P 5
#define R 3

int avail[R]={3,3,2};

int maxm[P][R]={
{7,5,3},{3,2,2},{9,0,2},
{2,2,2},{4,3,3}
};

int alloc[P][R]={
{0,1,0},{2,0,0},{3,0,2},
{2,1,1},{0,0,2}
};

int is_safe(int seq[P])
{
    int work[R],finish[P]={0},count=0;

    memcpy(work,avail,sizeof work);

    while(count<P)
    {
        int found=0;

        for(int p=0;p<P;p++)
        {
            if(finish[p]) continue;

            int ok=1;

            for(int r=0;r<R;r++)
                if(maxm[p][r]-alloc[p][r]>work[r])
                    ok=0;

            if(ok)
            {
                for(int r=0;r<R;r++)
                    work[r]+=alloc[p][r];

                finish[p]=1;
```

```
                seq[count++]=p;
                found=1;
            }
        }

        if(!found) return 0;
    }

    return 1;
}

void request(int p,int req[R])
{
    for(int r=0;r<R;r++)
    {
        if(req[r]>maxm[p][r]-alloc[p][r] ||
           req[r]>avail[r])
        {
            printf("P%d request: must wait (not available)\n",p);
            return;
        }
    }

    for(int r=0;r<R;r++)
    {
        avail[r]-=req[r];
        alloc[p][r]+=req[r];
    }

    int seq[P];

    if(is_safe(seq))
    {
        printf("P%d request: granted, safe sequence",p);

        for(int i=0;i<P;i++)
            printf(" P%d",seq[i]);

        printf("\n");
    }
    else
    {
        for(int r=0;r<R;r++)
        {
            avail[r]+=req[r];
            alloc[p][r]-=req[r];
        }

        printf("P%d request: denied (unsafe)\n",p);
    }
}

int main()
{
    int seq[P];
```

```
    if(is_safe(seq))
    {
        printf("initial state safe:");

        for(int i=0;i<P;i++)
            printf(" P%d",seq[i]);

        printf("\n");
    }

    request(1,(int[R]){1,0,2});
    request(4,(int[R]){3,3,0});
    request(0,(int[R]){0,2,0});

    return 0;
}
```

Output:

```
initial state safe: P1 P3 P4 P0 P2
P1 request: granted, safe sequence P1 P3 P4 P0 P2
P4 request: must wait (not available)
P0 request: denied (unsafe)
```

5. Why Is P0's Request Unsafe?

P0 requests (0, 2, 0).

After P1's request is granted, available resources become:

Available = (2, 3, 0)

After granting P0's request:

Available = (2, 1, 0)

No process can complete with these available resources because each process requires more resources than are currently available.

Therefore, the system enters an unsafe state and P0's request is denied.

B. Compare Deadlock Strategies

Strategy	Explanation	Advantage	Disadvantage
Prevention	Removes one necessary condition	Avoids deadlock	Reduces resource utilisation
Avoidance	Uses Banker's algorithm	Maintains safe state	Requires maximum resource demand

Detection and Recovery	Detects deadlock and recovers	Resources can be used efficiently	Recovery may lose work
Ignoring	Does not handle deadlock	Simple implementation	System may stop working

Evaluation: For a research centre, deadlock avoidance is useful because GPUs, storage and software licences are valuable shared resources.

C. Apply Banker's Algorithm (4 Marks)

Banker's algorithm has two main parts:

1. Safety Algorithm

- Calculate the Need matrix.
- Check whether available resources satisfy the process requirements.
- Execute the process if its needs are satisfied.
- Release its allocated resources.
- Continue until all processes finish.

2. Resource Request Algorithm

- Check whether the request is within the maximum need.
- Check available resources.
- Temporarily allocate resources.
- Run the safety algorithm.
- Grant the request if the state is safe; otherwise, deny it.

Need Formula:

\[Need = Max - Allocation \]

Initial Safe Sequence:

P1 → P3 → P4 → P0 → P2

Request Results:

Process	Request	Decision
P1	(1, 0, 2)	Granted
P4	(3, 3, 0)	Must wait
P0	(0, 2, 0)	Denied

D. Predict Before Implementing (1 Mark)

The time complexity of Banker's safety algorithm is:

\[\boxed{O(m \times n^2)} \]

Where:

- n = Number of processes
- m = Number of resource types

As the number of processes and resource types increases, execution time also increases.

Space complexity:

O(n \times m)

E. Measure (1 Mark)

Run the safety algorithm with different process counts.

Processes	Resource Types	Expected Growth
10	3	Very low execution time
100	3	Higher execution time
1000	3	Much higher execution time

Measurement: Use the C clock() function to record execution time on your computer.

```
#include <time.h>

clock_t start,end;

start=clock();
```

```
/* Call safety algorithm here */

end=clock();

printf("Time: %f seconds\n",
(double)(end-start)/CLOCKS_PER_SEC);
```

Note: Actual timings depend on your computer. Run the program and enter your measured values in the table.

F. Defend the Final Design (2 Marks)

Recommendation:

Deadlock avoidance using Banker's algorithm is recommended for the research centre.

Reasons:

1. It prevents the system from entering an unsafe state.
2. It improves resource allocation.
3. It avoids unnecessary process termination.
4. It ensures safe execution of processes.

Limitation: Banker's algorithm requires maximum resource requirements in advance and adds checking overhead for every request.

Conclusion: Banker's algorithm is suitable when resource requirements are known beforehand and system reliability is important.

MARK DISTRIBUTION

Question	Assessed Area	Bloom's Level	Marks
Q1	Processes and process states	K5 – Evaluate	6
Q2	CPU scheduling	K5 – Evaluate	8
Q3	Threads	K5 – Evaluate	6
Question	**Assessed Area**	**Bloom's Level**	**Marks**
Q4	Inter-process communication	K5 – Evaluate	6
Q5	Synchronisation: critical sections, semaphores and monitors	K5 – Evaluate	11
Q6	Day-4 Design Challenge – Deadlock handling and Banker's algorithm	K5 – Evaluate	13
	TOTAL		**50**

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your assignment report containing the required evidence for Q1–Q6
- C source code for all programs
- Gantt charts, diagrams, outputs and timings used as evidence

Assessment Verification

Students may be asked to explain any part of their design, code, calculations or results during a brief verification discussion/viva.

Overall Assessment Expectation

Model → Design → Implement → Measure → Compare → Evaluate → Recommend